



Color Clash Test Report

Color Clash - A Two-Player Fighting Game

TOBB ETU – BIL 482

April 2026

Team Members

Mehmet Umur ÖZÜ

Mehmet Yasin TOSUN

Mustafa GÖZÜTOK

1. Purpose of the Document

This report documents the final testing activities carried out for the Color Clash project. It presents the executed tests, the test environment, requirement traceability, result evidence, bug-fixing status, and deployment context for the tested build.

2. Test Scope

The report covers the final verification of normal match setup, real-time combat actions, mode switching, local tournament flow, winner/result displays, global shortcuts, input guards, and production build execution.

- Normal match initialization and player setup
- Movement, attack, collision, and damage behaviour
- Combat mode switching and special mode effects
- Tournament setup, round progression, and final podium
- Pause, restart, and return-to-setup controls
- Input guard while editing text/select fields
- Build smoke verification and automated logic checks

3. Test Responsibles

Responsible	Role
Mehmet Umur ÖZÜ	Developer, build verification, gameplay validation, regression review
Mehmet Yasin TOSUN	Developer, manual gameplay tester, tournament and UI validation
Mustafa GÖZÜTOK	Developer, combat logic tester, gameplay and interaction verification
Automated local smoke test script	Automated regression execution for selected logic and collision scenarios

4. Test Dates

Enter the actual execution dates before final submission. Dates should reflect the final test cycle used for the delivered version.

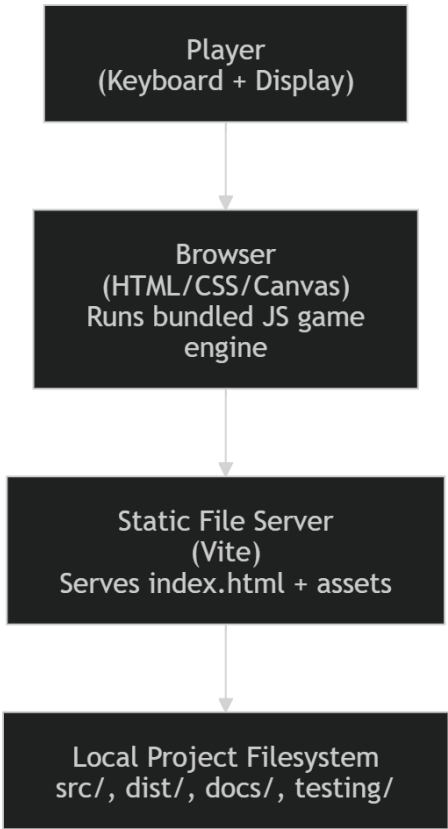
Activity	Date
Test Planning	10.04.2026
Automated Test Execution	14.04.2026
Manual Test Execution	14.04.2026

5. Test Configuration

Configuration Item	Value
Operating System	Windows (PowerShell)
Node.js Version	v24.13.0
npm Version	11.6.2
Application Stack	TypeScript + HTML5 Canvas + Vite
Build Command	npm.cmd run build
Automated Test Command	npm.cmd run test:smoke
Manual Test Environment	Latest Chrome or Edge browser
Execution Mode	Local development and local static build verification

6. Related Deployment Diagram

The tested system is a client-side browser application executed locally and served as static files during development/build validation.



Screenshot 1: Browser running the game

```
PS C:\Users\musta\Desktop\Dosyalarım\482> npm run dev

> fighting-game@1.0.0 dev
> vite

VITE v6.4.1 ready in 299 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

7. Requirement and Use Case Traceability

The following internal requirement identifiers are used to make the report directly traceable to the implemented features and to the executed test set.

Requirement ID	Related SRS Area	Description	Related Tests
FR-01	UC1	Start and initialize a normal match	TC-AUTO-001, TC-AUTO-002, TC-MAN-001
FR-02	UC2	Movement, attack, collision, and damage	TC-AUTO-004, TC-AUTO-005, TC-AUTO-007, TC-AUTO-008, TC-MAN-002
FR-03	UC3	Dynamic combat mode switching	TC-AUTO-003, TC-AUTO-006, TC-MAN-002
FR-04	UC4	Tournament registration and bracket progression	TC-MAN-004
FR-05	UC4 / Result UI	Winner and podium/result overlays	TC-MAN-003, TC-MAN-005
FR-06	Game Flow Control	Pause, restart, and return to setup	TC-MAN-006
FR-07	Input Handling	Ignore gameplay commands while editing fields	TC-MAN-007
NFR-01	Build / Deployment	Production build completes without error	TC-BUILD-001

NFR-02	Reliability	Stable execution in tested scenarios	All executed tests
--------	-------------	--------------------------------------	--------------------

8. Test Strategy and Method

A mixed strategy was used in the final verification cycle. Deterministic logic was covered by automated smoke tests, while full user flows and interaction-heavy behaviours were checked manually in the browser.

Method	Focus	Examples
Automated smoke tests	Deterministic logic and regression-sensitive behaviour	Factory outputs, command behaviour, damage logic, collision calculations
Manual functional tests	End-to-end gameplay and UI behaviour	Match start, controls, tournament progression, winner/podium displays
Build verification	Production readiness	Vite build completion and artifact generation

9. Execution Summary

Metric	Value	
Total Test Cases Executed	16	
Passed	16	
Failed	0	
Blocked	0	
Pass Rate	100%	
Test Type	Count	Result
Build Verification	1	PASS
Automated Functional / Smoke	8	PASS
Manual Functional / UI / Flow	7	PASS
Total	16	PASS

10. Detailed Test Cases and Results

The detailed results below are written in a compact format for clean Word-to-PDF conversion while preserving all required test information.

10.1 TC-BUILD-001 - Production Build Smoke Check

Field	Details
Related Requirement	NFR-01
Type	Automated
Priority	High
Test Date	[Insert date]
Tester	[Insert responsible person]
Preconditions	Dependencies are installed
Test Inputs	Run the production build command
Status	PASS
Steps	1. Run npm.cmd run build 2. Observe TypeScript and Vite build output
Expected Result	The build completes without error and production files are generated.
Actual Result	Build completed successfully and production artifacts were generated.
Evidence	Terminal output showing successful build completion.

Screenshot 2: Terminal output of npm run build

```
PS C:\Users\musta\Desktop\Dosyalarım\482> npm run build

> fighting-game@1.0.0 build
> tsc && vite build

vite v6.4.1 building for production...
✓ 30 modules transformed.
dist/index.html                20.65 kB | gzip:  4.25 kB
dist/assets/index-yoMTbdPv.js 54.19 kB | gzip: 14.24 kB
✓ built in 323ms
```

10.2 TC-AUTO-001 - Character Roster Consistency

Field	Details
Related Requirement	FR-01 / UC1
Type	Automated
Priority	High
Preconditions	Automated smoke test script is ready
Test Inputs	Direct CharacterFactory roster query
Status	PASS
Steps	1. Load CharacterFactory 2. Call getCharacterRoster()
Expected Result	Roster contains valid archetypes and expected mode pairs.
Actual Result	Roster size and mode mappings matched expected definitions.
Evidence	Automated smoke test output confirming correct roster exposure.

10.3 TC-AUTO-002 - Match Player Initialization

Field	Details
Related Requirement	FR-01 / UC1
Type	Automated

Priority	High
Preconditions	Automated smoke test environment is ready
Test Inputs	Character types: Fighter, Geomancer
Status	PASS
Steps	1. Create match players with selected archetypes 2. Verify IDs, spawn positions, and default modes
Expected Result	Player IDs, coordinates, and starting modes are initialized correctly.
Actual Result	IDs, positions, and initial modes matched expected values.
Evidence	Automated smoke test output confirming correct player initialization.

10.4 TC-AUTO-003 - Mode Switch Cycle

Field	Details
Related Requirement	FR-03 / UC3
Type	Automated
Priority	High
Preconditions	Automated smoke test environment is ready
Test Inputs	Switch command sequence
Status	PASS
Steps	1. Execute SwitchModeCommand 2. Repeat switch and verify cycle
Expected Result	Mode switches correctly and returns to the original cycle position.
Actual Result	Mode cycle behaviour matched expectation.
Evidence	Automated smoke test output confirming correct mode-cycle behaviour.

10.5 TC-AUTO-004 - Movement Command Flags

Field	Details
Related Requirement	FR-02 / UC2
Type	Automated
Priority	Medium
Preconditions	Automated smoke test environment is ready
Test Inputs	Simulated left key press and release
Status	PASS
Steps	1. Execute MoveCommand(left, true) 2. Execute MoveCommand(left, false)
Expected Result	Input flags update correctly for press/release.
Actual Result	Movement flags toggled correctly and moving state updated as expected.
Evidence	Automated smoke test output confirming command flag behaviour.

10.6 TC-AUTO-005 - Attack Hit Applies Damage

Field	Details
Related Requirement	FR-02 / UC2
Type	Automated
Priority	High
Preconditions	Players are placed within hitbox range
Test Inputs	Attack command with overlap positioning
Status	PASS
Steps	1. Place players within hitbox range 2. Trigger attack 3. Advance state update cycle
Expected Result	Defender health decreases after a successful hit.

Actual Result	Defender health dropped below the initial value.
Evidence	Automated smoke test output confirming damage application when overlap is present.

10.7 TC-AUTO-006 - Dark Absorb to Light Bonus Conversion

Field	Details
Related Requirement	FR-03 / UC3
Type	Automated
Priority	High
Preconditions	Automated Eclipse mode scenario is available
Test Inputs	Incoming damage = 30
Status	PASS
Steps	1. Switch Eclipse to Dark mode 2. Apply incoming damage 3. Recover from hit state 4. Switch to Light mode 5. Perform two Light attacks
Expected Result	First Light attack is boosted and the second returns to baseline.
Actual Result	First attack exceeded baseline and the second returned to expected base behaviour.
Evidence	Automated smoke test output confirming specialized mode conversion behaviour.

10.8 TC-AUTO-007 - Collision Midpoint Calculation

Field	Details
Related Requirement	FR-02 / UC2
Type	Automated
Priority	Medium
Preconditions	Overlap and non-overlap samples are available

Test Inputs	Rectangle A/B overlap and non-overlap samples
Status	PASS
Steps	1. Evaluate overlap sample 2. Evaluate non-overlap sample
Expected Result	Midpoint is returned for overlap; null is returned for non-overlap.
Actual Result	Returned values matched expected midpoint/null behaviour.
Evidence	Automated smoke test output confirming correct overlap point calculation.

10.9 TC-AUTO-008 - Character Collision Push-Apart

Field	Details
Related Requirement	FR-02 / UC2
Type	Automated
Priority	Medium
Preconditions	Overlapping player setup is available
Test Inputs	Player X coordinates: 400 and 430
Status	PASS
Steps	1. Create overlapping players 2. Run collision resolution 3. Compare horizontal distance before and after
Expected Result	Post-resolution distance becomes greater than pre-resolution distance.
Actual Result	Distance increased after collision resolution.
Evidence	Automated smoke test output confirming separation behaviour after resolution.

Screenshot 3: Terminal log showing all automated tests passed

```
> fighting-game@1.0.0 test:smoke
> node test/automated-smoke.test.cjs

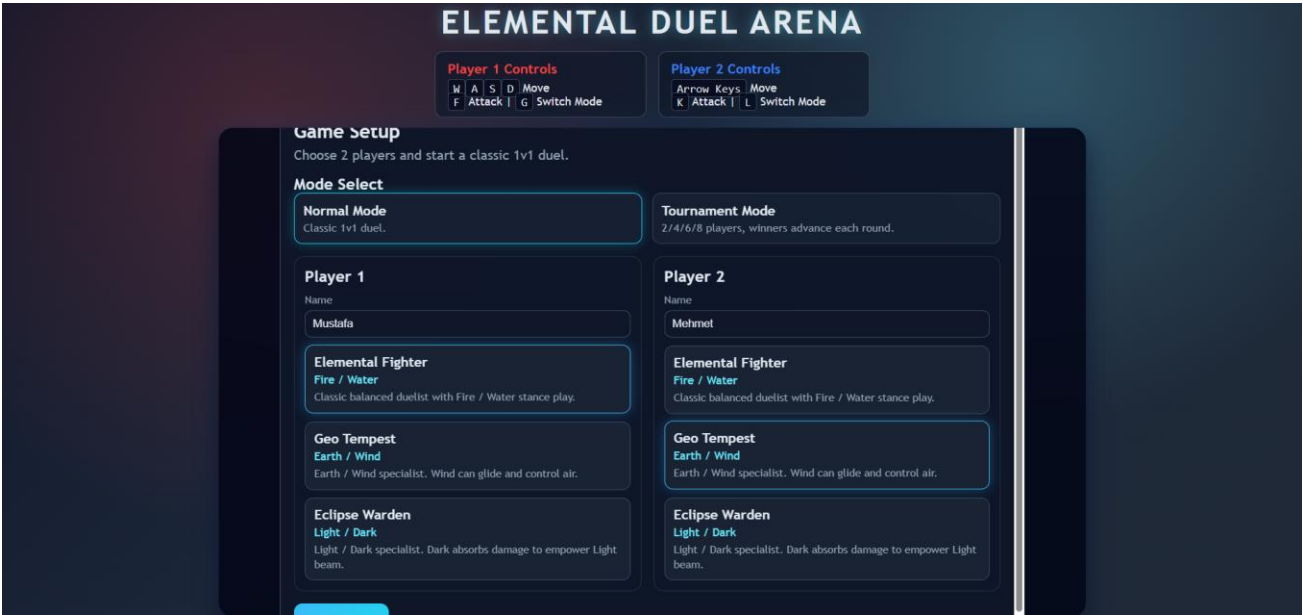
PASS: Character roster exposes expected archetypes and mode pairs
PASS: CharacterFactory creates match players with expected ids and spawn coordinates
PASS: SwitchModeCommand cycles character mode and returns to initial mode
PASS: MoveCommand updates movement flags and moving state
PASS: Attack state applies damage when opponent is inside hitbox
PASS: Dark mode absorbed damage is consumed as bonus by next Light attack
PASS: CollisionSystem detects overlap and computes overlap midpoint
PASS: CollisionSystem resolveCharacterCollision increases distance between overlapping players

Summary: 8 passed, 0 failed, 8 total
```

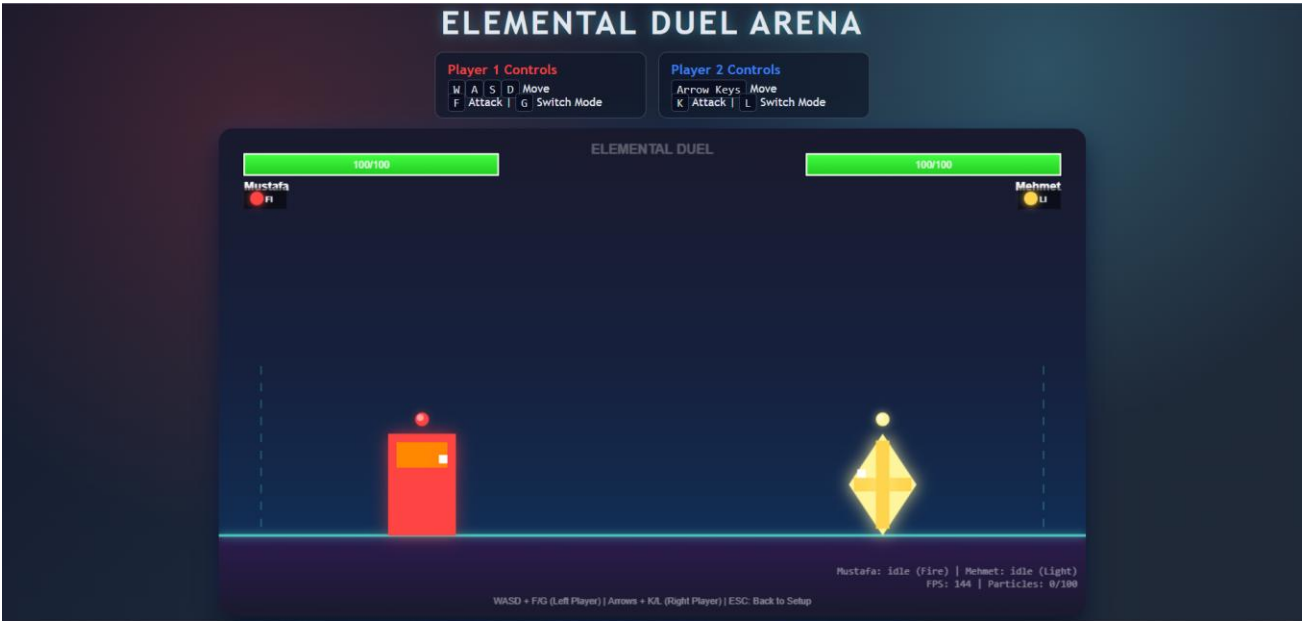
10.10 TC-MAN-001 - Normal Mode Setup and Start Flow

Field	Details
Related Requirement	FR-01 / UC1
Type	Manual
Priority	High
Preconditions	Browser session is active and development server is running
Test Inputs	Player names and character selections
Status	PASS
Steps	1. Choose normal mode 2. Set player names and archetypes 3. Press Start Match
Expected Result	Match starts successfully and selected setup values are reflected correctly.
Actual Result	Match started correctly and the setup summary reflected the selected names and archetypes.
Evidence	Manual browser execution by project tester.

Screenshot 4: Pre-match setup screen with names and character choices



Screenshot 5: Match started successfully

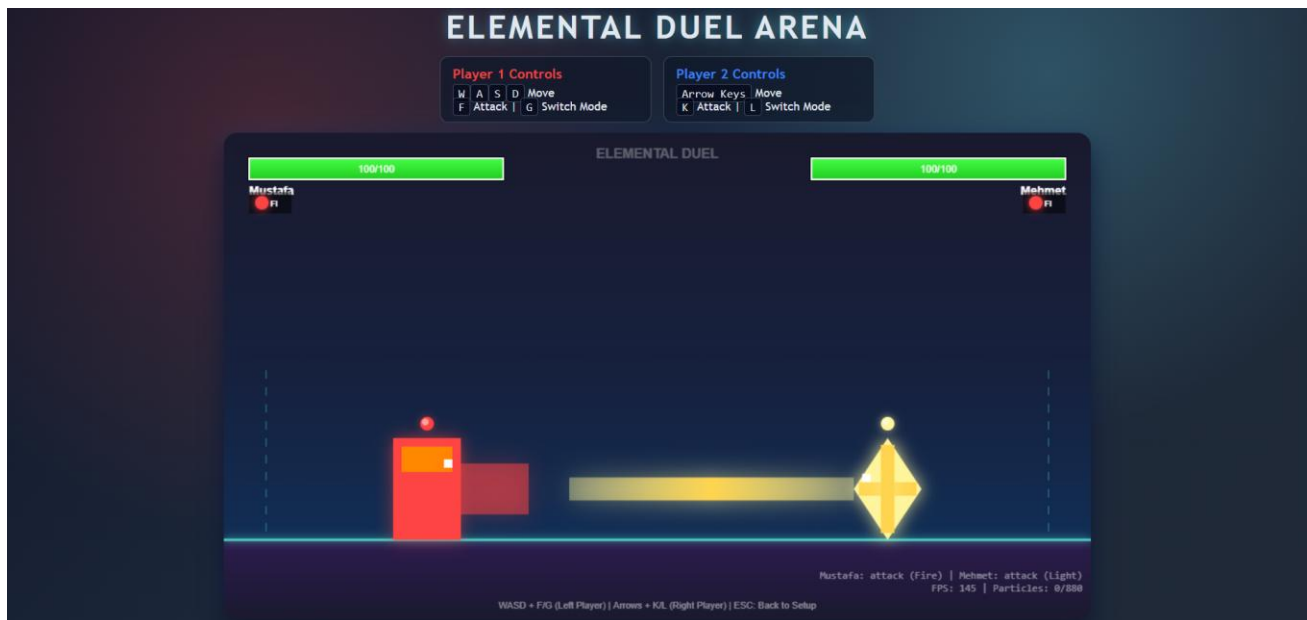


10.11 TC-MAN-002 - Keyboard Controls in Active Match

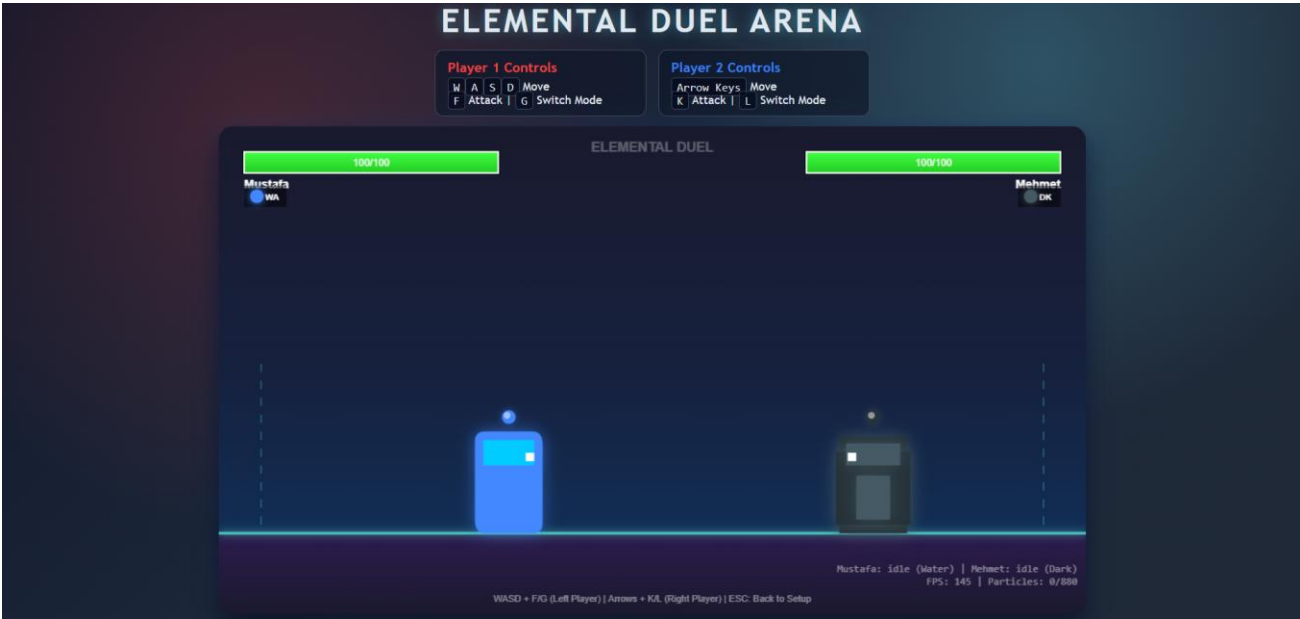
Field	Details
Related Requirement	FR-02, FR-03 / UC2, UC3
Type	Manual
Priority	High

Preconditions	An active match is running
Test Inputs	Gameplay keyboard actions
Status	PASS
Steps	1. Use WASD/F/G for Player 1 2. Use Arrow/K/L for Player 2
Expected Result	Movement, attack, and mode switch actions respond correctly for both players.
Actual Result	Player 1 and Player 2 controls responded correctly during manual execution.
Evidence	Manual browser execution by project tester.

Screenshot 6: Both characters moving or attacking in active match



Screenshot 7: Visible mode switch indicator change



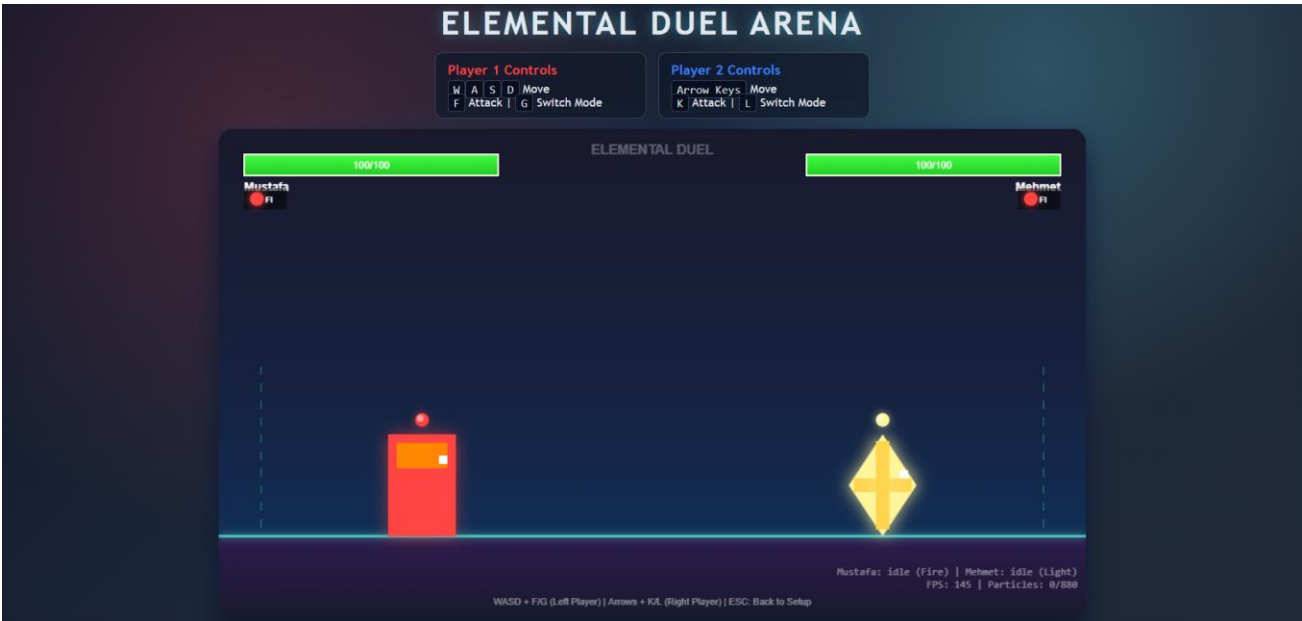
10.12 TC-MAN-003 - Winner Detection and Auto Restart

Field	Details
Related Requirement	FR-05
Type	Manual
Priority	High
Preconditions	A normal match is active
Test Inputs	Gameplay continued until one player is defeated
Status	PASS
Steps	1. Reduce one player health to zero 2. Observe winner overlay 3. Observe auto-restart timing
Expected Result	Winner is announced and match restarts automatically after a short delay.
Actual Result	Winner overlay appeared and auto-restart worked as expected.
Evidence	Manual browser execution by project tester.

Screenshot 8: Winner overlay



Screenshot 9: Restarted match or reset state after KO

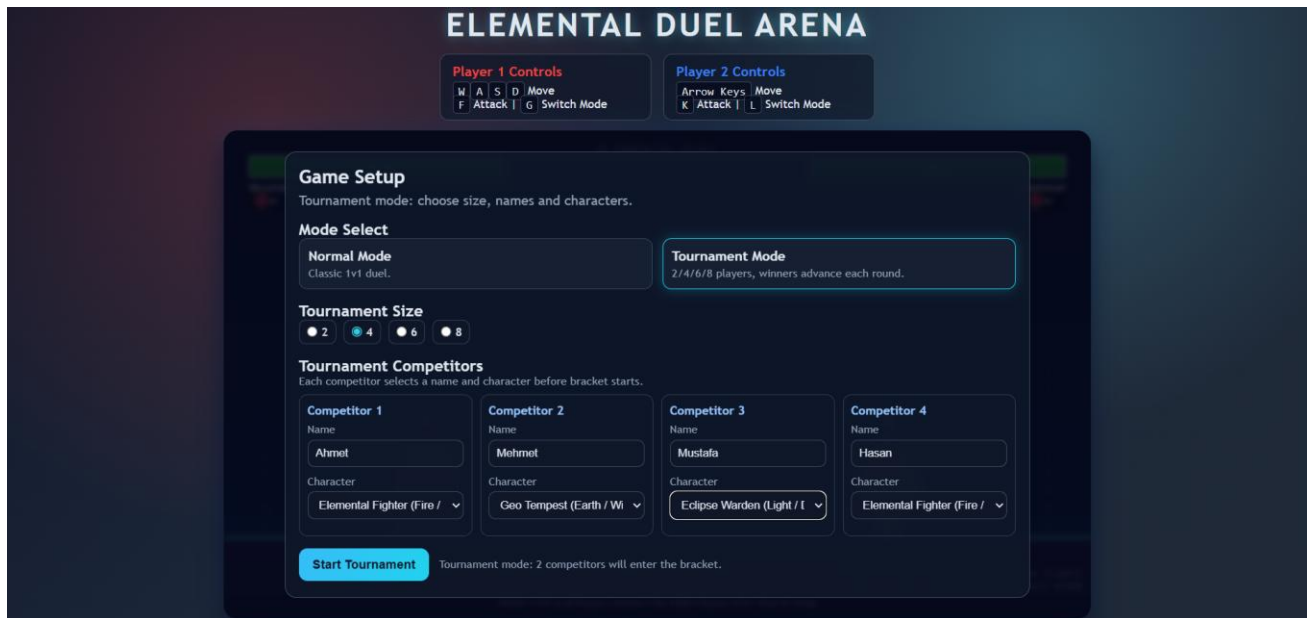


10.13 TC-MAN-004 - Tournament Setup and Round Progression

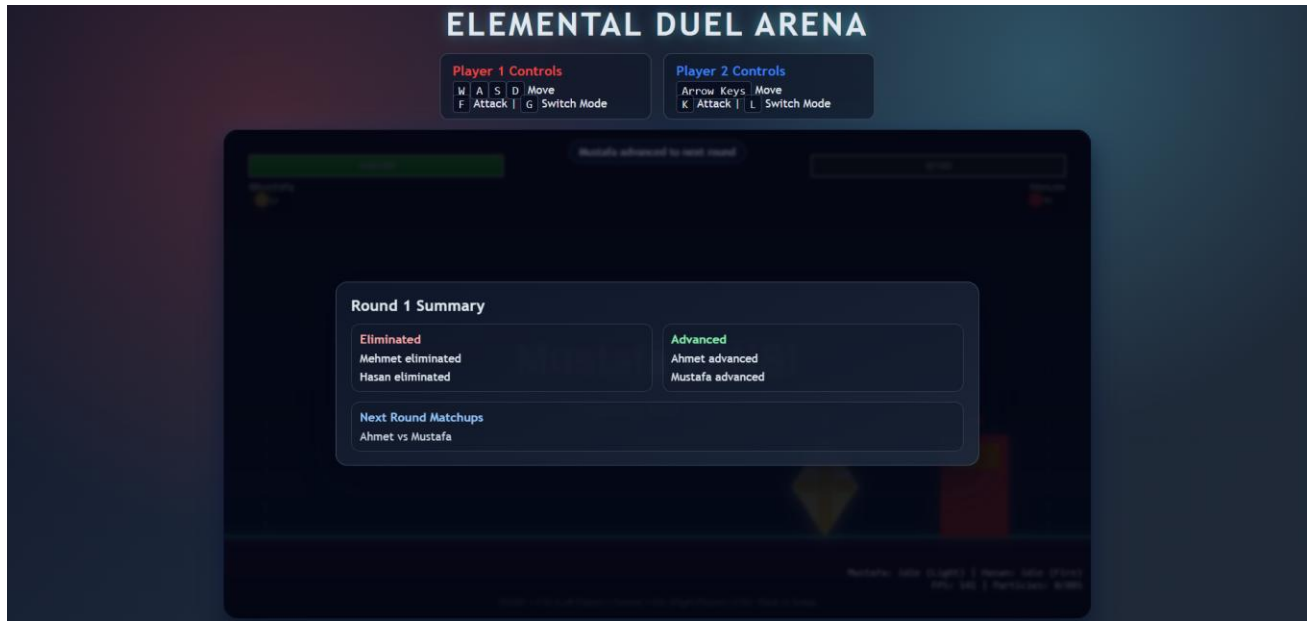
Field	Details
Related Requirement	FR-04 / UC4
Type	Manual
Priority	High

Preconditions	Browser session is active and tournament mode is selected
Test Inputs	Tournament size, participant names, selected characters
Status	PASS
Steps	1. Set tournament size and participant information 2. Start tournament 3. Observe sequential matches and round overlays
Expected Result	Bracket progresses sequentially and round progression overlay is shown.
Actual Result	Tournament bracket progressed sequentially and round overlays were shown correctly.
Evidence	Manual browser execution by project tester.

Screenshot 10: Tournament setup screen



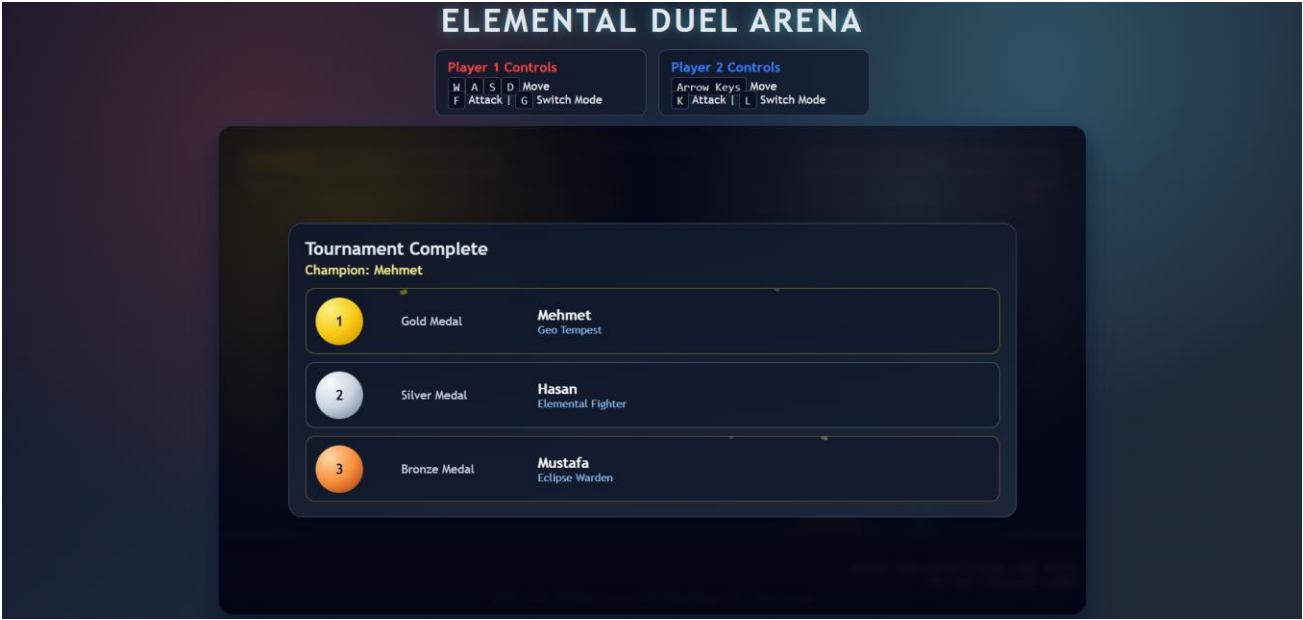
Screenshot 11: Bracket or round progression overlay



10.14 TC-MAN-005 - Final Podium and Champion Summary

Field	Details
Related Requirement	FR-05 / UC4
Type	Manual
Priority	High
Preconditions	Tournament has been completed
Test Inputs	Final match outcome
Status	PASS
Steps	1. Complete tournament final 2. Observe final podium and result summary
Expected Result	1st, 2nd, and 3rd place are visible together with champion summary.
Actual Result	Final podium and champion subtitle were displayed correctly.
Evidence	Manual browser execution by project tester.

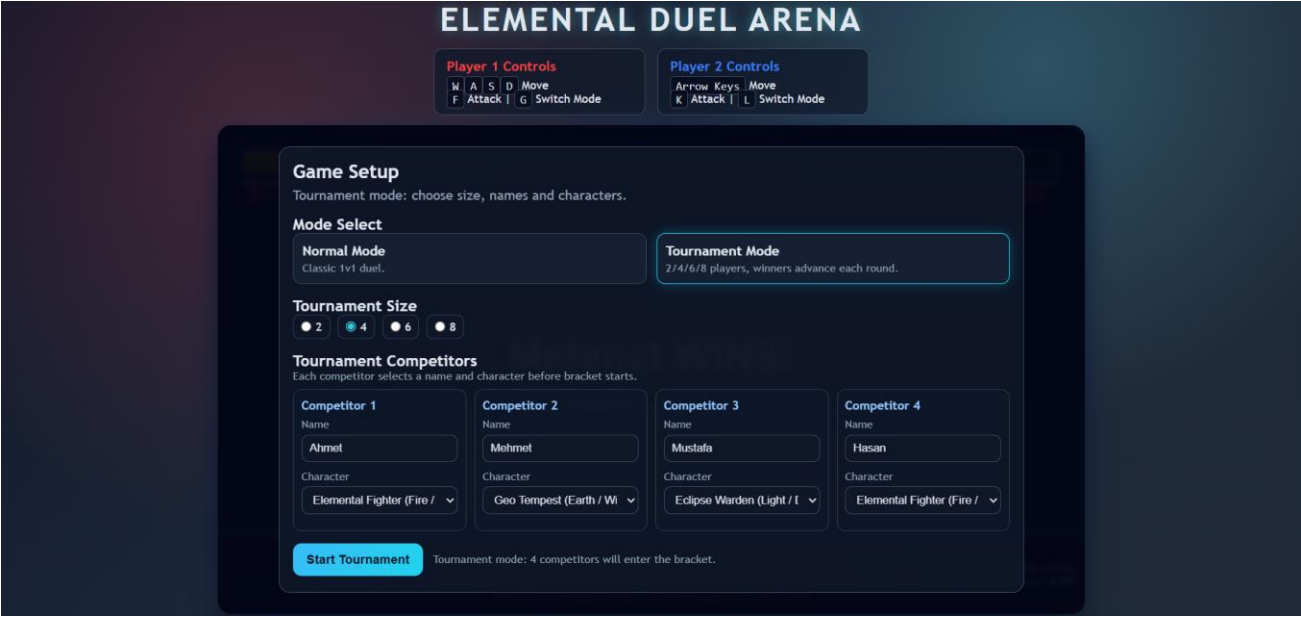
Screenshot 12: Final podium screen



10.15 TC-MAN-006 - Global Shortcut (Esc)

Field	Details
Related Requirement	FR-06
Type	Manual
Priority	Medium
Preconditions	Gameplay session is active
Test Inputs	Shortcut key presses
Status	PASS
Steps	1. Press Esc to return to setup
Expected Result	Esc returns to setup.
Actual Result	Tested shortcut behaviour worked as expected.
Evidence	Manual browser execution by project tester.

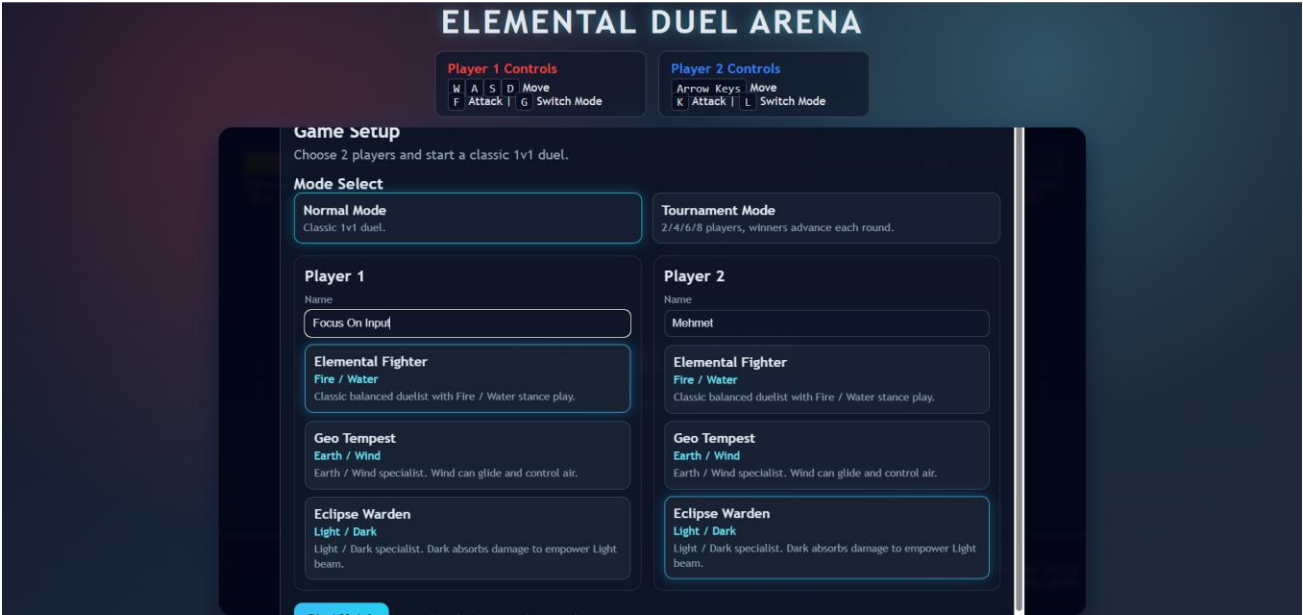
Screenshot 13: Setup screen shown after Esc



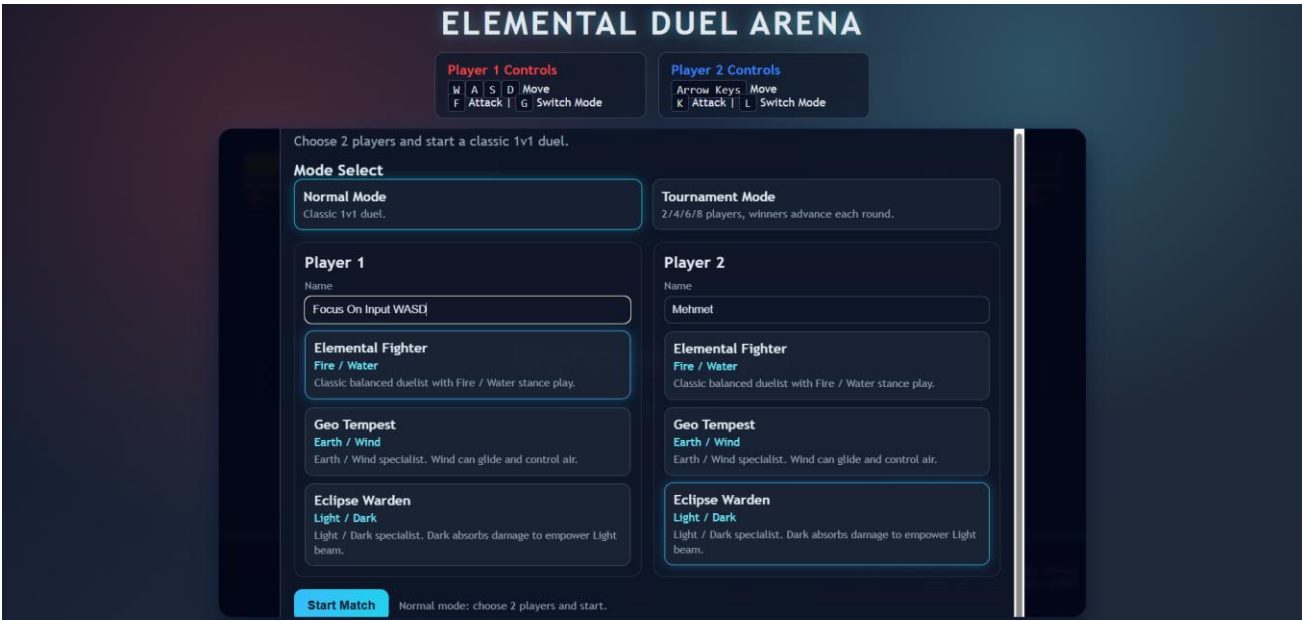
10.16 TC-MAN-007 - Input Guard While Editing Fields

Field	Details
Related Requirement	FR-07
Type	Manual
Priority	Medium
Preconditions	Cursor is focused in an input or select element
Test Inputs	Typing and gameplay keys while field is focused
Status	PASS
Steps	1. Focus name input or selection field 2. Press gameplay keys
Expected Result	Gameplay input must be ignored while editing focused form elements.
Actual Result	Gameplay commands were ignored while a form input was focused.
Evidence	Manual browser execution by project tester.

Screenshot 14: Cursor focused in input field



Screenshot 15: No unintended gameplay action while field is focused



11. Recording of Test Results

The final PDF should include visual evidence and terminal output excerpts. The placeholders below indicate the minimum screenshot set required for a complete submission package.

Appendix ID	Required Evidence
Appendix A1	Build output (terminal screenshot)
Appendix A2	Automated smoke test output (terminal

	screenshot)
Appendix A3	Normal match setup evidence
Appendix A4	Active gameplay evidence
Appendix A5	Winner overlay evidence
Appendix A6	Tournament setup evidence
Appendix A7	Tournament result / podium evidence
Appendix A8	Pause / restart / return-to-setup evidence
Appendix A9	Input guard evidence

12. Testing and Bug Fixing Discussion

Testing was performed iteratively to verify both core gameplay logic and player-facing behaviour. Automated checks focused on deterministic logic such as factory outputs, command execution, strategy/state effects, collision detection, and damage application. Manual checks focused on end-to-end user flows such as starting a match, playing through a combat session, progressing through a tournament, and observing result overlays and shortcut behaviour.

During the final test cycle used for this report, no reproducible critical or medium-severity defect was encountered. Accordingly, no corrective bug-fix implementation was required specifically for the final reporting cycle. Instead, the executed test set was used as a confirmation and regression verification pass for the delivered version.

13. Fixed Bugs Record

No reproducible bug requiring corrective implementation was recorded during the final test cycle. The table below is intentionally marked as not applicable for this cycle while preserving the format expected in the final deliverable.

Cycle	Bug Record Status	Verification Basis	Conclusion
Final verification cycle	No reproducible defect requiring fix was encountered	All executed final tests passed	No corrective change required in this cycle

14. Defect Retest / Regression Statement

Even though no new bug-fix entry was created during the final cycle, the executed test suite served as a regression verification pass. The completed run showed 16 executed tests and 16 PASS results, which supports the conclusion that the delivered version remained stable in the tested scenarios.

15. Risks, Limitations, and Notes

- Manual screenshot evidence must still be inserted before final PDF export.
- Dates must be filled with the actual execution dates.
- The report reflects the tested local environment and does not claim exhaustive multi-device/browser certification.
- If any issue is discovered after this version, the fixed-bug table should be updated instead of leaving it as N/A.

16. Final Evaluation

Based on the recorded results, the tested version of Color Clash passed build verification, automated smoke validation, and manual validation for match setup, combat behaviour, tournament progression, result display, and game flow controls. The project is therefore considered acceptable for final demonstration, provided that the marked screenshot placeholders and real execution dates are completed before submission.